A Change Stream Ledger for MongoDB: Tamper Evidence Without a Ledger Engine

COMP.8157 Advanced Database Topics, Fall 2026: D.2.1 Project Full Proposal

Olumide Ogunjebi
School of Computer Science
University of Windsor, Windsor, Canada
ogunjeb@uwindsor.ca

Rohith Nelluri
School of Computer Science
University of Windsor, Windsor, Canada
nellurir@uwindsor.ca

Yadu Pratap
School of Computer Science
University of Windsor, Windsor, Canada
Singh8a4@uwindsor.ca

Abstract. **Ledger databases let an organization prove that stored records were never silently altered, yet widely used document databases offer no such guarantee. MongoDB has no native ledger, and Amazon QLDB, a managed ledger built around documents, reached end of support in 2025. This project instead retrofits tamper evidence onto an unmodified MongoDB replica set through two public interfaces. A synchronous design writes each document change and its hash in the same multi document transaction, while an asynchronous design reads committed changes from change streams, groups them into Merkle blocks, chains the blocks, and publishes digests to an external log. A verifier checks the chain and compares every current document with the state the ledger implies. We will measure throughput, latency, storage, and the window between commit and digest on YCSB, SmallBank, and a financial sample dataset, compare both designs with the native Ledger of SQL Server 2022 under identical workloads, and run a scripted insider attack suite. The expected contributions are an open source ledger layer for MongoDB, a measured comparison of retrofit and engine ledgers, and an attack detection matrix.**

Index Terms. **tamper evidence, ledger databases, MongoDB, change streams, SQL Server Ledger, database forensics**

I. Introduction and Motivation

Records of payments, prescriptions, and audit events are useful as evidence only if their integrity can be demonstrated to a third party. Access control alone cannot provide that assurance, because the people with the greatest power to change data, database administrators and anyone who steals their credentials, are precisely the people such controls trust. Ledger databases close this gap by hashing every change into a chain anchored by a digest held outside the database, so that any later alteration becomes detectable. Industry and research have produced several such systems, including SQL Ledger in Azure SQL Database and SQL Server [1], LedgerDB at Alibaba [2], and research prototypes such as GlassDB [3].

The problem is timely for two reasons. First, Amazon QLDB, the managed ledger built around a document data model, reached end of support on 31 July 2025, and its users were directed to a relational service that, as one industry report notes, does not keep a permanent and immutable record of changes [4]. Second, MongoDB, a widely deployed document database and the one our course uses, offers no cryptographic tamper evidence at all. An organization running MongoDB today can choose between a commercial service that places its own layer in the write path, such as ProvenDB [5], and forensic techniques that compare database states after an incident [6]. Neither produces a verifiable record of every committed change.

Existing systems fall short for a structural reason. Every ledger in the research literature obtains its guarantees by owning the storage or execution layer: purpose built engines [2], [3], modifications to an existing engine [1], [7], [8], trusted execution environments [9], [10], or authenticated structures pushed into the storage engine itself [11]. The most systematic benchmark of verifiable databases, VeriBench [12], measured six systems, every one of them purpose built or reimplemented for the study. None of this work helps an organization that must keep its existing document database and cannot alter its engine. Yet MongoDB already maintains a totally ordered logical log of writes, the oplog, which replicas use to stay synchronized [13], and it exposes the majority committed portion of that log through change streams [14]. This project asks whether a log that exists for availability can also serve as the backbone of a verifiable ledger.

II. Problem Statement

We consider a MongoDB replica set that stores a set of protected collections and is operated by an administrator who may act maliciously or whose credentials may be stolen. The adversary can issue arbitrary commands, write directly to any collection including those of the ledger, restart the server in other modes, stop our services, and restore backups, but cannot alter digests once they have been published to an external append only log. Under this model, the problem is to construct a ledger, using only the public interfaces of an unmodified MongoDB server, that satisfies three properties. Completeness requires that every majority committed change to a protected collection be recorded with its before and after images. Tamper evidence requires that any insertion, modification, or deletion of a protected document or ledger entry that is inconsistent with the published digests be detected by a verifier with read access to the database, the ledger, and the digest log. Bounded cost requires that the overhead on throughput, latency, and storage be small enough, and measured precisely enough, for an organization to decide whether to deploy it.

The problem rests on core database principles. Completeness depends on the semantics of the oplog and on the distinction between acknowledged and majority committed

writes, since writes acknowledged at weaker write concerns can be rolled back after a failover [13]. Tamper evidence depends on atomicity, because a document and its ledger entry must commit together or not at all, and on isolation, because concurrent transactions that extend the same chain conflict with one another. Bounded cost is a question of logging overhead, storage amplification from retained before images, and the price of verification scans. We frame the evaluation as three research questions: what each design costs relative to an unprotected database and to a native engine ledger (RQ1), how long a committed change remains outside the published digests under realistic load (RQ2), and which insider attacks each design detects and with what forensic evidence (RQ3).

III. Related Work

A. Closest Related Work

The closest work is SQL Ledger [1], now available in Azure SQL Database and in every edition of SQL Server 2022. Each row changed by a transaction is hashed, transaction hashes are combined into Merkle trees, the resulting blocks are chained, earlier row versions are kept in history tables, and database digests are stored outside the database so that a verifier can recompute the chain. On SQL Server, automatic digest upload is supported only to Azure storage, so an on premises deployment must generate its digests itself [15]. Our ledger deliberately follows this structure, which lets SQL Ledger serve as a fair reference, but the project is not a reproduction. SQL Ledger is built into a relational engine; we obtain comparable guarantees for documents from outside an unmodified engine, compare a synchronous and an asynchronous variant that SQL Ledger does not offer, and evaluate all three configurations against attacks rather than on performance alone.

B. Comparative Positioning

Purpose built ledgers own the whole stack. LedgerDB [2] anchors a centralized ledger with timestamp authorities and batch accumulated Merkle trees, and GlassDB [3] authenticates database states rather than transactions so that proofs stay compact, with independent auditors checking the service. VeDB [10] argues that Merkle based ledgers pay a high price in CPU and I/O and replaces them with a verifiable Shrubs array and a trusted execution component, reporting insertion up to 70 times faster than Merkle structures, while LETUS [11] pushes the authenticated data structure down into the storage engine of a blockchain platform for similar reasons. These systems move verification deeper into the engine to gain performance. Our project moves it in the opposite direction, entirely out of the engine, and asks what that portability costs. We therefore build Merkle trees per block rather than per write, limiting the cost that VeDB identifies.

Other work modifies an engine or depends on hardware. Nathan et al. [7] added about 4,000 lines of C to PostgreSQL to create a blockchain relational database for mutually distrustful organizations, Xia et al. [8] combine authenticated data structures with deterministic, batch based concurrency control so that clients can verify the ACID properties of an outsourced database, and VeriDB [9] runs query processing inside an Intel SGX enclave. Snodgrass et al. [16] were among the first to hash transactions and notarize digests externally to expose tampering with audit logs, through an extension built into MySQL with Berkeley DB. Each result requires engine code or hardware that a typical MongoDB operator cannot deploy, and none targets a document store.

VeriBench [12] is the most relevant evaluation study. It compares continuous and deferred verification across six systems and shows that batching deferred work raises throughput, which motivates our asynchronous design, yet every system it measures is purpose built or reimplemented, its SQL Ledger variant omits the SQL layer, and it does not test attacks. For MongoDB specifically, ProvenDB [5] offered a service on MongoDB Atlas that hashes documents into Merkle trees and anchors the roots on the Bitcoin blockchain, but it is a commercial layer in the write path and we found no published evaluation of its overhead. Chopade and Pachghare [6] detect tampering by comparing old and new database states, which supports investigation after an incident but gives no proof for individual changes. Table I summarizes where each approach places verifiability.

Table I
Where Prior Work Places Verifiability

Work	Year	Mechanism	Engine unchanged	Trusted element
SQL Ledger [1]	2021	Inside SQL Server	No	Digest storage
LedgerDB [2]	2020	Purpose built ledger	No	Time authority
GlassDB [3]	2023	Purpose built ledger	No	Auditors
VeDB [10]	2023	Shrubs array, TEE	No	TEE
LETUS [11]	2024	ADS in storage	No	Blockchain
Nathan [7]	2019	Modified PostgreSQL	No	Peer nodes
Litmus [8]	2022	Proof based DBMS	No	Client verifier
VeriDB [9]	2021	SGX enclave	No	Intel SGX
VeriBench [12]	2023	Benchmark	Rebuilt	Varies
ProvenDB [5]	2019	MongoDB service	Yes	Bitcoin anchor
Chopade [6]	2021	State comparison	Yes	None
This project	**2026**	**Change streams**	**Yes**	**Digest log**

IV. Novelty and Contributions

What assumption are we breaking? The literature assumes, almost without exception, that tamper evidence must be implemented where the data is stored: inside a purpose built engine, inside a modified engine, inside trusted hardware, or, in the most recent work, inside the storage layer itself [10], [11]. We break that assumption by treating the replication log of an unmodified database as sufficient raw material for a ledger. The assumption matters because it decides who can have verifiable data. If it holds, every organization that cannot change or replace its database engine is excluded. If it fails, verifiability becomes a portable service that can be attached to an existing deployment.

The project makes four contributions. First, a ledger design for stock MongoDB in two variants: a synchronous variant that hashes inside the writing transaction and spreads contention

over partitioned chain heads, and an asynchronous variant that builds Merkle blocks from change streams, records gaps in the log explicitly instead of hiding them, and publishes digests to an external log. Second, to our knowledge, the first measured comparison of retrofit ledgers with a production engine ledger, SQL Server Ledger, under identical workloads and digest intervals. Third, an attack matrix reporting, for each insider attack, whether each configuration detects it, how quickly, and what forensic evidence survives, which no prior benchmark provides. Fourth, a reproducible kit of container configurations, workloads, and scripts published on GitHub. The novelty lies not in a new hash structure but in where verifiability is placed, and in evidence about what that placement costs.

V. System Design and Algorithm Overview

Fig. 1 shows the architecture. A single node MongoDB 8.0 replica set holds the protected collections and a separate ledger database. Clients, driven by YCSB or by our SmallBank implementation, write either directly or through the ledger library. Design S lives in that library, whereas Design A is an independent service that consumes change streams; both produce the same ledger format, so a single verifier serves both. A SQL Server 2022 instance runs the same workloads against updatable ledger tables and provides the native baseline.

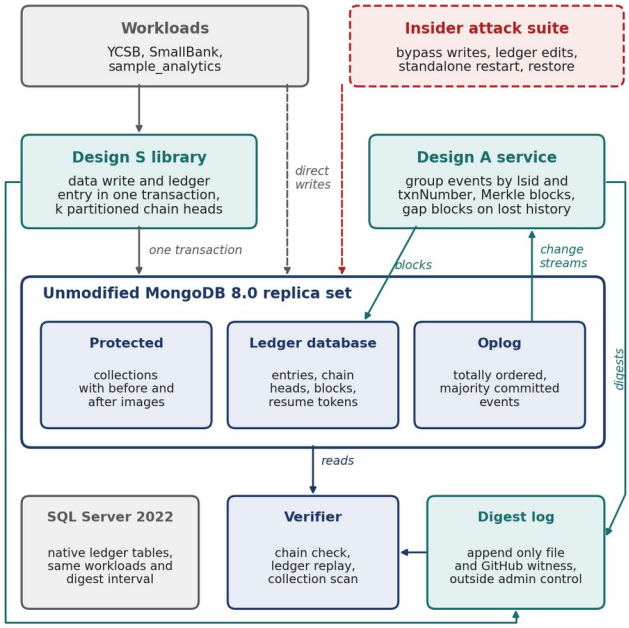


Fig. 1. System architecture. Design S writes data and ledger entries in one transaction; Design A builds Merkle blocks from change streams; both publish digests to an external log that the verifier checks. SQL Server Ledger runs the same workloads as a native baseline.

Every protected document is hashed after canonicalization: field names are sorted recursively and values are serialized in MongoDB Extended JSON canonical mode, so that logically equal documents with different field orders produce the same SHA256 hash. A ledger entry records the namespace, the document key, the operation type, the hashes of the before and after images, a transaction identifier, and the commit timestamp. Entries are the leaves of a Merkle tree per transaction, and transaction roots are the leaves of a Merkle tree per block, following the hashing rules of RFC 9162, which prefix leaves and interior nodes differently to resist second preimage attacks [17]. Each block stores its root, the hash of the previous block, its first and last log positions, and the change stream resume token at which it ends; the block hash is the digest that is published.

In Design S, the library wraps each application operation in a multi document transaction committed with majority write concern. Inside the transaction it reads the before image, applies the write, computes the entry, reads one of k chain heads selected by hashing the document key, appends the entry linked to that head, and advances the head. A transaction that advances a head conflicts with any concurrent transaction on the same head, so MongoDB aborts one of them with a transient error and the driver retries it; varying k from 1 to 16 measures how far partitioning relieves this contention. A background task periodically folds the k heads into a block and publishes its digest. Design S therefore keeps the ledger current at commit, but it sees only writes that pass through the library.

Design A opens a change stream on the protected database with before and after images required. For each event it (1) canonicalizes and hashes both images, (2) assigns the event to its transaction through the lsid and txnNumber fields, which MongoDB 8.0 documents as the way to identify the events of one transaction [18], treating a write outside any transaction as its own group, (3) closes a block when the digest interval expires and computes the transaction and block roots, (4) stores the block together with its resume token in one write, and (5) publishes the block hash to an append only digest file that is also pushed to a separate GitHub repository acting as an external witness. Because change streams report only majority committed writes, a write that is later rolled back never enters the ledger. If the service restarts and its resume token has already left the oplog, the missing events cannot be recovered; rather than failing silently, the service appends a gap block that records the last and first available log positions, so that auditors see exactly where coverage was lost.

The verifier performs three checks. It recomputes every block hash and confirms that the chain matches the published digests and contains no unexplained gaps; it replays the ledger to derive the expected latest hash of every protected document; and it scans each protected collection, hashing every document and reporting modified, missing, and unexpected documents. The scan is linear in the size of the data, as in earlier audit log validators [16]. For the baseline, SQL Server stores the same data in updatable ledger tables, generates digests with its built in procedure at the same interval as our ledger, and checks them with its built in verification procedure [15]. Table II lists the attack suite with the outcome we expect for each configuration; the evaluation will confirm or refute these expectations.

TABLE II
PLANNED ATTACKS AND EXPECTED DETECTION

Attack	Design S	Design A	SQL Ledger
Write that skips the library	Mismatch only	Recorded	Recorded
Edit ledger entries or history	Detected	Detected	Detected
Standalone restart, then write	Mismatch only	Mismatch only	Not applicable
Oplog rolls over while down	Not affected	Gap block	Not applicable
Restore an older backup	Detected	Detected	Detected
Tamper before next digest	Missed	Missed	Missed

VI. IMPLEMENTATION PLAN

All components will be written in Python 3.12, using the pymongo driver for MongoDB, pyodbc for SQL Server, hashlib for hashing, and matplotlib for plots. MongoDB 8.0 Community Server will run as a single node replica set in Docker, with the oplog size and minimum retention period set so that a stopped service can catch up within an experiment; SQL Server 2022 will run in Docker in its free Developer edition, with the Express edition as a fallback. All software is free, and the complete environment runs on one laptop. The code, hosted in the GitHub repository the course requires, comprises five components: a ledger core library for canonical hashing, Merkle trees, blocks, and digests; the Design S client library; the Design A change stream service; the verifier; and an experiment harness that drives workloads, injects attacks, and collects metrics. Unit tests will confirm that hashing is deterministic and that every Merkle proof verifies.

Three workloads will be used. YCSB [19] supplies standard key value mixes through its MongoDB and JDBC bindings, and we will run its update heavy workload A and read mostly workload B over one million records. SmallBank [20] models short banking transactions that touch several accounts at once; we will implement it in Python for both systems so that multi document transactions are exercised. MongoDB's sample_analytics dataset, which models accounts, customers, and transactions for a financial services application and can be restored locally with mongorestore, supplies realistic nested documents [21]. The configurations are an unprotected baseline, Design S with k of 1, 4, and 16, Design A with digest intervals of 1, 5, and 30 seconds, and SQL Server with and without ledger tables, each run at 1 to 32 client threads and repeated five times, with medians and interquartile ranges reported.

Work is divided by component. Olumide Ogunjebi, who brings prior training in cybersecurity and digital forensics, leads the threat model, the attack suite, and the verifier; Rohith Nelluri leads the ledger core, Design S, and the SQL Server baseline; and Yadu Pratap leads the Design A change stream service, the workloads, and the analysis of results. All three members share testing and documentation and will present the proposal together. The team will work in two week Scrum sprints tracked in Hive, with every change reviewed through a pull request on GitHub.

VII. RISKS AND MITIGATION

The main technical risk is losing events in Design A. If the service falls behind and the oplog or the stored before images expire, the events cannot be recovered. We will size the oplog and its minimum retention generously, monitor the lag between the newest oplog entry and the last processed event, and record any loss as a gap block so that coverage is never overstated. A second risk is contention in Design S: if every transaction extends the same head, throughput may collapse under concurrency. Partitioned heads and driver retries mitigate this, and if contention remains severe we will report it as a finding rather than conceal it.

A third risk is nondeterministic hashing, since logically equal documents can differ in field order or numeric type after a round trip through a driver, which would produce false alarms. Canonical serialization with sorted keys and explicit type information, together with tests that rehash documents read back from the server, addresses this. A fourth risk concerns measurement validity: running two database servers and a benchmark on one laptop invites interference, so we will run one system at a time with fixed container resource limits, warm up every run, repeat it, and report variation. Because SQL Server Express caps memory and processor use, all performance comparisons will use the Developer edition.

Finally, there are risks of scope and interpretation. The threat model excludes an adversary who controls the digest log, and a change made and concealed before its digest is published cannot be detected by any configuration, SQL Ledger included; we will state this limit explicitly and quantify the window instead of claiming absolute protection. To protect the schedule, we will build Design A and the verifier first, since they carry the core result, and treat partitioned heads and the GitHub witness as extensions that can be dropped if time runs short.

VIII. TIMELINE AND MILESTONES

Table III aligns the work with the course schedule. Each phase ends with a concrete, demonstrable deliverable. The midterm review on 8 November will include first measurements from both designs, leaving the final weeks for the full experiments, the attack suite, and documentation.

TABLE III
PHASES, DATES, AND DELIVERABLES

Phase	Dates	Main work	Deliverable
1	5 to 11 Oct	Docker testbed, GitHub, data	D.2.2 talk (6 Oct), testbed
2	12 to 25 Oct	Ledger core, Design S, verifier	Design S passing tests
3	26 Oct to 8 Nov	Design A, SQL Server baseline	D.3 report, first results
4	9 to 22 Nov	Experiments and attack suite	Result tables and plots
5	23 to 29 Nov	Demo, design and user guides	D.4 demo (29 Nov)
6	30 Nov to 9 Dec	Final report and presentation	D.5 (1 Dec), D.6 (9 Dec)

REFERENCES

[1] P. Antonopoulos, R. Kaushik, H. Kodavalla, S. Rosales Aceves, R. Wong, J. Anderson, and J. Szymaszek, “SQL Ledger: Cryptographically verifiable data in Azure SQL Database,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, doi: 10.1145/3448016.3457558.

[2] X. Yang, Y. Zhang, S. Wang, B. Yu, F. Li, Y. Li, and W. Yan, "LedgerDB: A centralized ledger database for universal audit and verification," *Proc. VLDB Endow.*, vol. 13, no. 12, 2020, doi: 10.14778/3415478.3415540.

[3] C. Yue, T. T. A. Dinh, Z. Xie, M. Zhang, G. Chen, B. C. Ooi, and X. Xiao, "GlassDB: An efficient verifiable ledger database system through transparency," *Proc. VLDB Endow.*, vol. 16, no. 6, 2023, doi: 10.14778/3583140.3583152.

[4] R. Losio, "AWS discontinues Amazon Quantum Ledger Database (QLDB)," *InfoQ*, Jul. 2024. [Online]. Available: https://www.infoq.com/news/2024/07/aws-kill-qldb

[5] J. Vaughan, "ProvenDB brings blockchain applications to world of MongoDB," *TechTarget SearchDataManagement*, May 2019. [Online]. Available: https://www.techtarget.com/data-technologies/news/252462934/ProvenDB-brings-blockchain-applications-to-world-of-MongoDB

[6] R. Chopade and V. Pachghare, "Performance analysis of proposed database tamper detection technique for MongoDB," in *Proc. 5th Int. Congr. Information and Communication Technology (ICICT)*, Springer, 2021, doi: 10.1007/978-981-15-5856-6_39.

[7] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran, "Blockchain meets database: Design and implementation of a blockchain relational database," *Proc. VLDB Endow.*, vol. 12, no. 11, 2019, doi: 10.14778/3342263.3342632.

[8] Y. Xia, X. Yu, M. Butrovich, A. Pavlo, and S. Devadas, "Litmus: Towards a practical database management system with verifiable ACID properties and transaction correctness," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2022, doi: 10.1145/3514221.3517851.

[9] W. Zhou, Y. Cai, Y. Peng, S. Wang, K. Ma, and F. Li, "VeriDB: An SGX-based verifiable database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, doi: 10.1145/3448016.3457308.

[10] X. Yang, R. Zhang, C. Yue, Y. Liu, B. C. Ooi, Q. Gao, Y. Zhang, and H. Yang, "VeDB: A software and hardware enabled trusted relational database," *Proc. ACM Manag. Data*, vol. 1, Art. 194, 2023, doi: 10.1145/3589774.

[11] S. Tian, Z. Lu, H. Zhuo, X. Tang, P. Hong, S. Chen, D. Yang, Y. Yan, Z. Jiang, H. Zhang, and G. Jiang, "LETUS: A log-structured efficient trusted universal blockchain storage," in *Companion of the Int. Conf. Management of Data (SIGMOD)*, 2024, doi: 10.1145/3626246.3653390.

[12] C. Yue, M. Zhang, C. Zhu, G. Chen, D. Loghin, and B. C. Ooi, "VeriBench: Analyzing the performance of database systems with verifiability," *Proc. VLDB Endow.*, vol. 16, no. 9, 2023, doi: 10.14778/3598581.3598588.

[13] W. Schultz, T. Avitabile, and A. Cabral, "Tunable consistency in MongoDB," *Proc. VLDB Endow.*, vol. 12, no. 12, 2019.

[14] MongoDB, Inc., "Change streams," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/changeStreams/

[15] Microsoft, "Digest management," *SQL Server Documentation*, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ledger-digest-management

[16] R. T. Snodgrass, S. S. Yao, and C. Collberg, "Tamper detection in audit logs," in *Proc. 30th Int. Conf. Very Large Data Bases (VLDB)*, 2004.

[17] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency version 2.0," IETF RFC 9162, Dec. 2021.

[18] MongoDB, Inc., "Update event," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/reference/change-events/update/

[19] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Computing (SoCC)*, 2010, doi: 10.1145/1807128.1807152.

[20] M. Alomari, M. Cahill, A. Fekete, and U. Röhm, "The cost of serializability on platforms that use snapshot isolation," in *Proc. IEEE 24th Int. Conf. Data Engineering (ICDE)*, 2008.

[21] MongoDB, Inc., "Load sample data into a local deployment," *MongoDB Manual v8.0*. [Online]. Available: https://www.mongodb.com/docs/v8.0/sample-data/load-sample-data-local/